ILineUpOptions

+summaryHeader: boolean
+animated: boolean
+expandLineOnHover: boolean
+sidePanel: boolean
+sidePanelCollapsed: boolean
+hierarchyIndicator: boolean
+onDialogBackgroundClick: "cancel" | "confirm"
+livePreviews: Partial<ILivePreviewOptions>
+defaultSlopeGraphMode: "item" | "band"
+labelRotation: number
+rowHeight: number
+rowPadding: number
+groupHeight: number
+groupPadding: number
+levelOfDetail: (rowIndex: number) => "high" | "low"
+dynamicHeight: (data: (IGroupItem | Readonly<IOrderedGroup>)[], ranking: Ranking) => IDynamicHeight
+customRowUpdate: (row: HTMLElement, rowIndex: number) => void
+toolbarActions: IToolbarLookup<IToolbarAction>
+toolbarDialogAddons: IToolbarLookup<IToolbarDialogAddon>
+resolveToolbarActions: (col: Column, keys: string[], lookup: IToolbarLookup<IToolbarAction>) => IToolbarAction[]
+resolveToolbarDialogAddons: (col: Column, keys: string[], lookup: IToolbarLookup<IToolbarDialogAddon>) => IToolbarDialogAddon[]
+renderers: { [type: string]: ICellRendererFactory
}
+canRender: (type: string, renderer: ICellRendererFactory, col: Column, mode: ERenderMode) => boolean
+flags: Partial<ILineUpFlags>
+ignoreUnsupportedBrowser: boolean
+copyableRows: boolean
+instanceId: string
+selectionActivateFilter: string | boolean

ITaggleOptions

+overviewMode: boolean

ILineUpLike

+node: HTMLElement
+data: IDataProvider

+dump(): any
+destroy(): void

DataBuilder

-columns: (IColumnDesc | ((data: any[]) => IColumnDesc))[]
- providerOptions: Partial<ILocalDataProviderOptions & IDataProviderOptions>
- rankBuilders: ((data: ADataProvider) => void)[]
-_deriveColors: boolean
-data: Record<string, unknown>[]

+scheduledTaskExecutor(): this
+showTopN(): this
+aggregationStrategy(): this
+propagateAggregationState(): this
+singleSelection(): this
+multiSelection(): this
+filterGlobally(): this
+deriveColumns(): this
+deriveColors(): this
+registerColumnType(): this
+column(): this
+restore(): this
+defaultRanking(): this
+ranking(): this
+buildData(): LocalDataProvider
+build(): LineUp
+buildTaggle(): Taggle

LineUpBuilder

#options: Partial<ITaggleOptions>

+animated(): this
+livePreviews(): this
+labelRotation(): this
+sidePanel(): this
+hierarchyIndicator(): this
+defaultSlopeGraphMode(): this
+summaryHeader(): this
+expandLineOnHover(): this
+overviewMode(): this
+ignoreUnsupportedBrowser(): this
+registerRenderer(): this
+canRender(): this
+registerToolbarAction(): this
+registerToolbarDialogAddon(): this
+rowHeight(): this
+groupRowHeight(): this
+dynamicHeight(): this
+disableAdvancedRankingFeatures(): this
+disableAdvancedModelFeatures(): this
+disableAdvancedUIFeatures(): this
+instanceId(): this
+selectionActivateFilter(): this
+copyableRows(): this

AEventDispatcher

- listeners: Dispatch<any>
- listenerEvents: Set<string>
-forwarder: (...args: any[]) => void

+on(): this
#listenersChanged(): void
#createEventList(): string[]
#fire(): void
-fireImpl(): void
#forward(): void
#unforward(): void

IEventHandler

+on(): this

LazyFilter<T>

-_length: number
- it: any
-filters: ((v: T, i: number) => boolean)[]

+filter(): ISequence<T>
+map(): ISequence<U>
+forEach(): void
+__computed(): { next: () => any
}
+some(): boolean
+every(): boolean
+reduce(): U

ALazyMap<T,T2>

#it: any

+filter(): ISequence<T2>
+abstract map(): ISequence<U>
#abstract mapV(): T2
+forEach(): void
+__computed(): { next: () => { value: T2
done: boolean
}
}
+some(): boolean
+every(): boolean
+reduce(): U

LazyMap1<T1,T2>

#map12: (v: T1, i: number) => T2

#mapV(): T2
+map(): ISequence<U>

LazyMap2<T1,T2,T3>

- map12: (v: T1, i: number) => T2
- map23: (v: T2, i: number) => T3

+map(): ISequence<U>
#mapV(): T3

LazyMap3<T1,T2,T3,T4>

- map12: (v: T1, i: number) => T2
- map23: (v: T2, i: number) => T3
- map34: (v: T3, i: number) => T4

+map(): ISequence<U>
#mapV(): T4

LazySeq<T>

-_arr: any
- iterable: Iterable<T>

+__computed(): any
+filter(): LazyFilter<T>
+map(): LazyMap1<T, U>
+forEach(): void
+some(): boolean
+every(): boolean
+reduce(): U

ConcatSequence<T>

- seqs: ISequence<ISequence<T>>

+__computed(): { next: () => { value: T
done: boolean
}
}
+filter(): ISequence<T>
+map(): ISequence<U>
+forEach(): void
+some(): boolean
+every(): boolean
+reduce(): U

IForEachAble<T>

+forEach(): void

ISequence<T>

+length: number

+filter(): ISequence<T>
+map(): ISequence<U>
+some(): boolean
+every(): boolean
+reduce(): U

IWorkerMessage

+type: string
+uid: number

IStatsWorkerMessage

+refIndices: string
+indices?: UIntTypedArray
+refData: string
+data?: UIntTypedArray | Float32Array | Int32Array | Float64Array | readonly string[]

IBoxPlotData

+min: number
+max: number
+median: number
+q1: number
+q3: number
+outlier?: number[]
+whiskerLow?: number
+whiskerHigh?: number

IAdvancedBoxPlotData

+mean: number
+missing: number
+count: number
+kdePoints: { v: number
p: number
}[]

IHistogramStats<T>

+min: T
+max: T
+missing: number
+count: number
+maxBin: number
+hist: readonly IBin<T>[]

IStatistics

+mean: number

IDateStatistics

+histGranularity: IDateHistGranularity

OrderedSet<T>

+__computed: any
-set: any
- list: T[]

+clear(): void
+addAll(): this
+add(): this
+has(): any
+delete(): boolean
+deleteAll(): boolean
+forEach(): void
+__computed(): any

Iterable

ActionColumn

+actions: IAction[]
+groupActions: IGroupAction[]

+getLabel(): string
+getValue(): string
+compare(): number

Column

+static FLAT_ALL_COLUMNS: -1
+static EVENT_WIDTH_CHANGED: "widthChanged"
+static EVENT_LABEL_CHANGED: "labelChanged"
+static EVENT_METADATA_CHANGED: "metaDataChanged"
+static EVENT_DIRTY: "dirty"
+static EVENT_DIRTY_HEADER: "dirtyHeader"
+static EVENT_DIRTY_VALUES: "dirtyValues"
+static EVENT_DIRTY_CACHES: "dirtyCaches"
+static EVENT_RENDERER_TYPE_CHANGED: "rendererTypeChanged"
+static EVENT_GROUP_RENDERER_TYPE_CHANGED: "groupRendererChanged"
+static EVENT_SUMMARY_RENDERER_TYPE_CHANGED: "summaryRendererChanged"
+static EVENT_VISIBILITY_CHANGED: "visibilityChanged"
- uid: string
-width: number
+parent: Readonly<IColumnParent>
- metadata: Readonly<IColumnMetaData>
- renderer: string
-groupRenderer: string
-summaryRenderer: string
-visible: boolean
+desc: Readonly<IColumnDesc>

+assignNewId(): void
#createEventList(): string[]
+on(): this
+getWidth(): number
+hide(): void
+show(): void
+isVisible(): boolean
+getVisible(): boolean
+setVisible(): void
+flatten(): number
+setWidth(): void
+setWidthImpl(): void
+setMetaData(): void
+getMetaData(): Readonly<IColumnMetaData>
+sortByMe(): boolean
+groupByMe(): boolean
+isGroupedBy(): number
+toggleMySorting(): boolean
- isSortedByMeImpl(): { asc: "asc" | "desc"
priority: number
}
+isSortedByMe(): { asc: "asc" | "desc"
priority: number
}
+groupSortByMe(): boolean
+toggleMyGroupSorting(): boolean
+isGroupSortedByMe(): { asc: "asc" | "desc"
priority: number
}
+removeMe(): boolean
+attach(): void
+detach(): void
+insertAfterMe(): boolean
+insertBeforeMe(): boolean
+nextSibling(): Column
+previousSibling(): Column
+findMyRanker(): Ranking
+dump(): any
+restore(): void
+getLabel(): string
+getValue(): any
+getExportValue(): any
+getColor(): string
+toCompareValue(): ICompareValue | ICompareValue[]
+toCompareValueType(): ECompareValueType | ECompareValueType[]
+group(): IGroup
+toCompareGroupValue(): ICompareValue | ICompareValue[]
+toCompareGroupValueType(): ECompareValueType | ECompareValueType[]
+isFiltered(): boolean
+clearFilter(): boolean
+filter(): boolean
+getRenderer(): string
+getGroupRenderer(): string
+getSummaryRenderer(): string
+setRenderer(): void
+setGroupRenderer(): void
+setSummaryRenderer(): void
+markDirty(): void
+getHeaderLabel(): { content: string
asHTML: boolean
}
+getSummaryLabel(): { content: string
asHTML: boolean
}

AggregateGroupColumn

+static EVENT_AGGREGATE: "aggregate"

#createEventList(): string[]
+on(): this
+isAggregated(): false | EAggregationState
+setAggregated(): boolean

IAggregateGroupColumnDesc

+isAggregated(): EAggregationState
+setAggregated(): void IColumnDesc
+label: string
+labelAsHTML?: boolean | IStyleHTMLWrapper
+type: string

AnnotateColumn

+static EVENT_VALUE_CHANGED: "valueChanged"
- annotations: any

#createEventList(): string[]
+on(): this
+getValue(): any
+dump(): any
+restore(): void
+setValue(): boolean

StringColumn

+static EVENT_FILTER_CHANGED: "filterChanged"
+static EVENT_GROUPING_CHANGED: "groupingChanged"
- static FILTER_MISSING: "__FILTER_MISSING"
-currentFilter: IStringFilter
+alignment: EAlignment
+escape: boolean
-currentGroupCriteria: IStringGroupCriteria

#createEventList(): string[]
+on(): this
+getValue(): string
+getLabel(): string
+dump(): any
+restore(): void
+isFiltered(): boolean
+filter(): any
+getFilter(): IStringFilter
+setFilter(): void
+clearFilter(): boolean
+getGroupCriteria(): IStringGroupCriteria
+setGroupCriteria(): void
+group(): IGroup
+toCompareValue(): string
+toCompareValueType(): ECompareValueType
+toCompareGroupValue(): string
+toCompareGroupValueType(): ECompareValueType

ArrayColumn<T>

-_dataLength: number
- originalLabels: string[]

+getValue(): T[]
+getValues(): T[]
+getLabels(): string[]
+getLabel(): string
+getMap(): { key: string
value: T
}[]
+getMapLabel(): { key: string
value: string
}[]

IArrayColumnDesc<T>

ValueColumn<T>

+static EVENT_DATA_LOADED: "dataLoaded"
+static RENDERER_LOADING: "loading"
- accessor: (row: IDataRow, desc: Readonly<IValueColumnDesc<T>>) => T
- loaded: boolean

+onDataUpdate(): void
#createEventList(): string[]
+on(): this
+getLabel(): string
+getRaw(): T
+getValue(): T
+isLoaded(): boolean
+setLoaded(): void
+getRenderer(): string
+dump(): any
+restore(): void

IArrayColumn<T>

+labels: string[]
+dataLength: number

+getLabels(): string[]
+getValues(): T[]

IArrayDesc

+dataLength?: number
+labels?: string[]

IValueColumnDesc<T>

+lazyLoaded?: boolean

+accessor(): T

BooleanColumn

+static EVENT_FILTER_CHANGED: "filterChanged"
+static EVENT_COLOR_MAPPING_CHANGED: "colorMappingChanged"
+static GROUP_TRUE: { name: string
color: string
}
+static GROUP_FALSE: { name: string
color: string
}
+static DEFAULT_TRUE_VALUES: any[]
-currentFilter: ICategoricalFilter
-colorMapping: ICategoricalColorMappingFunction
+categories: ICategory[]

#createEventList(): string[]
+on(): this
+getValue(): boolean
+getCategoryOfBoolean(): ICategory
+getCategory(): ICategory
+getCategories(): ICategory[]
+iterCategory(): ICategory[]
+getColor(): any
+getLabel(): any
+getLabels(): any
+getValues(): any
+getMap(): any
+getMapLabel(): any
+getSet(): any
+dump(): any
+restore(): void
+getColorMapping(): ICategoricalColorMappingFunction
+setColorMapping(): void
+isFiltered(): boolean
+filter(): any
+getFilter(): any
+setFilter(): void
+clearFilter(): boolean
+toCompareValue(): number
+toCompareValueType(): ECompareValueType
+group(): any

ICategoricalColumn

+getCategory(): ICategory

BooleansColumn

+static EVENT_COLOR_MAPPING_CHANGED: "colorMappingChanged"
- colorMapping: ICategoricalColorMappingFunction

+getSet(): any
+toCompareValue(): number
+toCompareValueType(): ECompareValueType
+getCategories(): { name: string
label: string
color: string
value: number
}[]
+iterCategory(): { name: string
label: string
color: string
value: number
}[]
+getColors(): string[]
#createEventList(): string[]
+on(): this
+getColorMapping(): ICategoricalColorMappingFunction
+setColorMapping(): any
+dump(): any
+restore(): void

ISetColumn

+getSet(): Set<ICategory>

BoxPlotColumn

+static EVENT_MAPPING_CHANGED: "mappingChanged"
+static EVENT_COLOR_MAPPING_CHANGED: "colorMappingChanged"
+static EVENT_SORTMETHOD_CHANGED: "sortMethodChanged"
+static EVENT_FILTER_CHANGED: "filterChanged"
- numberFormat: (n: number) => string
-sort: ESortMethod
- mapping: IMappingFunction
-colorMapping: IColorMappingFunction
- original: Readonly<IMappingFunction>
- deriveMapping: readonly boolean[]
- currentFilter: INumberFilter

+onDataUpdate(): void
+getNumberFormat(): (n: number) => string
+toCompareValue(): number
+toCompareValueType(): ECompareValueType
+getBoxPlotData(): IBoxPlotData
+getRange(): [string, string]
+getRawBoxPlotData(): IBoxPlotData
+getRawValue(): IBoxPlotData
+getExportValue(): any
+getValue(): IBoxPlotData
+getNumber(): number
+getRawNumber(): number
+iterNumber(): number[]
+iterRawNumber(): number[]
+getLabel(): string
+getSortMethod(): ESortMethod
+setSortMethod(): void
+dump(): any
+restore(): void
#createEventList(): string[]
+on(): this
+getOriginalMapping(): IMappingFunction
+getMapping(): IMappingFunction
+setMapping(): void
+getColor(): any
+getColorMapping(): IColorMappingFunction
+setColorMapping(): void
+isFiltered(): any
+getFilter(): INumberFilter
+setFilter(): void
+filter(): any
+clearFilter(): any

IBoxPlotDesc

+sort?: ESortMethod

IBoxPlotColumn

+getBoxPlotData(): IBoxPlotData
+getMapping(): IMappingFunction
+getRawBoxPlotData(): IBoxPlotData
+getSortMethod(): string
+setSortMethod(): void

INumberDesc

+numberFormat?: string
+filterAccuracy?: number
+showMinimumRepresentation?: boolean

ReplacementColorMappingFunction

+map: ReadonlyMap<string, string>

+apply(): any
+toJSON(): { type: string
map: any
}
+clone(): ReplacementColorMappingFunction
+eq(): boolean
+static restore(): ReplacementColorMappingFunction

ICategoricalColorMappingFunction

+apply(): string
+toJSON(): ITypedDump
+clone(): ICategoricalColorMappingFunction
+eq(): boolean

CategoricalColumn

+static EVENT_FILTER_CHANGED: "filterChanged"
+static EVENT_COLOR_MAPPING_CHANGED: "colorMappingChanged"
+categories: ICategory[]
- colorMapping: ICategoricalColorMappingFunction
- lookup: any
- currentFilter: ICategoricalFilter
- categoryOrder: "given" | "small-to-large" | "large-to-small" | ((categories: readonly ICategory[]) => ICategory[])

+onDataUpdate(): void
- sortCategories(): void
#createEventList(): string[]
+on(): this
+getValue(): any
+getCategory(): any
+getLabel(): any
+getCategories(): any[]
+getValues(): boolean[]
+getLabels(): string[]
+getMap(): { key: string
value: boolean
}[]
+getMapLabel(): { key: string
value: string
}[]
+getSet(): any
+iterCategory(): any[]
+dump(): any
+restore(): void
+getColor(): string
+getColorMapping(): ICategoricalColorMappingFunction
+setColorMapping(): void
+isFiltered(): boolean
+filter(): boolean
+getFilter(): any
+setFilter(): void
+clearFilter(): boolean
+toCompareValue(): number
+toCompareValueType(): ECompareValueType
+group(): IGroup
+toCompareGroupValue(): ICompareValue[]
+toCompareGroupValueType(): ECompareValueType[]
+getGroupRenderer(): string

CategoricalMapColumn

+static EVENT_COLOR_MAPPING_CHANGED: "colorMappingChanged"
+categories: ICategory[]
- lookup: any
- colorMapping: ICategoricalColorMappingFunction

+onDataUpdate(): void
#createEventList(): string[]
+on(): this
- parseValue(): any
+getCategoryMap(): { key: string
value: any
}[]
+getCategories(): any[]
+getColors(): { key: string
value: string
}[]
+getValue(): { key: string
value: any
}[]
+getLabels(): { key: string
value: any
}[]
+getColorMapping(): ICategoricalColorMappingFunction
+setColorMapping(): any
+dump(): any
+restore(): void
+iterCategory(): any[]

MapColumn<T>

+getValue(): IKeyValue<T>[]
+getLabels(): IKeyValue<string>[]
+getMap(): IKeyValue<T>[]
+getMapLabel(): IKeyValue<string>[]
+getLabel(): string

ICategoricalLikeColumn

+categories: ICategory[]

+getColorMapping(): ICategoricalColorMappingFunction
+setColorMapping(): void
+iterCategory(): IForEachAble<ICategory>
+getCategories(): ICategory[]

CategoricalsColumn

+static EVENT_COLOR_MAPPING_CHANGED: "colorMappingChanged"
+static EVENT_FILTER_CHANGED: "filterChanged"
+categories: ICategory[]
- lookup: any
- colorMapping: ICategoricalColorMappingFunction
- currentFilter: ISetCategoricalFilter

+onDataUpdate(): void
#createEventList(): string[]
+on(): this
+getCategories(): any[]
+getColors(): string[]
+iterCategory(): any[]
+getValues(): any[]
+getLabels(): any[]
+getColorMapping(): ICategoricalColorMappingFunction
+setColorMapping(): any
+isFiltered(): boolean
+filter(): boolean
+getFilter(): ISetCategoricalFilter
+setFilter(): void
+clearFilter(): boolean
+toCompareValue(): any[]
+toCompareValueType(): ECompareValueType[]
+dump(): any
+restore(): void

SequentialColorFunction

+static FUNCTIONS: { [key: string]: (v: number) => string
}
+apply: (v: number) => string
+name: string

+toJSON(): string
+clone(): this
+eq(): boolean

DivergentColorFunction

+static FUNCTIONS: { [key: string]: (v: number) => string
}
+apply: (v: number) => string
+name: string

+toJSON(): string
+clone(): this
+eq(): boolean

UnknownColorFunction

+apply: (v: number) => string

+toJSON(): string
+clone(): this
+eq(): boolean

SolidColorFunction

+color: string

+apply(): string
+toJSON(): string
+clone(): this
+eq(): boolean

QuantizedColorFunction

+base: IColorMappingFunction
+steps: number

+apply(): string
+toJSON(): { type: string
base: string | ITypedDump
steps: number
}
+clone(): QuantizedColorFunction
+eq(): boolean

CustomColorMappingFunction

-scale: any
+entries: { value: number
color: string
}[]

+apply(): any
+toJSON(): { type: string
entries: { value: number
color: string
}[]
}
+clone(): CustomColorMappingFunction
+eq(): boolean

IColorMappingFunction

+apply(): string
+toJSON(): string | ITypedDump
+clone(): IColorMappingFunction
+eq(): boolean

CompositeColumn

+static EVENT_FILTER_CHANGED: "filterChanged"
+static EVENT_ADD_COLUMN: "addColumn"
+static EVENT_MOVE_COLUMN: "moveColumn"
+static EVENT_REMOVE_COLUMN: "removeColumn"
#_children: Column[]

#createEventList(): string[]
+on(): this
+assignNewId(): void
+flatten(): number
+dump(): any
+restore(): void
+insert(): Column
+move(): Column
#insertImpl(): Column
#moveImpl(): Column
+push(): Column
+at(): Column
+indexOf(): number
+insertAfter(): Column
+moveAfter(): Column
+insertBefore(): Column
+moveBefore(): Column
+remove(): boolean
#removeImpl(): boolean
+isFiltered(): boolean
+clearFilter(): boolean
+filter(): boolean
+isLoaded(): boolean

IColumnParent

+fqid: string
+fqpath: string

+remove(): boolean
+insert(): Column
+insertBefore(): Column
+insertAfter(): Column
+move(): Column
+moveBefore(): Column
+moveAfter(): Column
+findMyRanker(): Ranking
+indexOf(): number
+at(): Column

CompositeNumberColumn

- numberFormat: (n: number) => string

+getNumberFormat(): (n: number) => string
+getLabel(): string
+getValue(): number
#compute(): number
+getNumber(): number
+getRawNumber(): number
+iterNumber(): number[]
+iterRawNumber(): number[]
+getExportValue(): any
+toCompareValue(): any
+toCompareValueType(): any
+toCompareGroupValue(): any
+toCompareGroupValueType(): any
+getRenderer(): string

ICompositeNumberDesc

+numberFormat?: string

INumberColumn

+getNumber(): number
+getRawNumber(): number
+iterNumber(): IForEachAble<number>
+iterRawNumber(): IForEachAble<number>
+getNumberFormat(): (v: number) => string

DateColumn

+static EVENT_FILTER_CHANGED: "filterChanged"
+static EVENT_GROUPING_CHANGED: "groupingChanged"
+static DEFAULT_DATE_FORMAT: "%x"
-format: (date: Date) => string
- parse: (date: string) => Date
-currentFilter: INumberFilter
-currentGrouper: IDateGrouper

+getFormatter(): (date: Date) => string
+dump(): any
+restore(): void
#createEventList(): string[]
+on(): this
+getValue(): Date
+getDate(): Date
+iterDate(): Date[]
+getLabel(): string
+isFiltered(): boolean
+clearFilter(): boolean
+getFilter(): INumberFilter
+setFilter(): void
+filter(): boolean
+toCompareValue(): number
+toCompareValueType(): ECompareValueType
+getDateGrouper(): any
+setDateGrouper(): void
+group(): IGroup
+toCompareGroupValue(): number
+toCompareGroupValueType(): ECompareValueType

IDateColumn

+getFormatter(): (v: Date) => string
+getDate(): Date
+iterDate(): IForEachAble<Date>
+getFilter(): INumberFilter
+setFilter(): void

DatesColumn

+static EVENT_SORTMETHOD_CHANGED: "sortMethodChanged"
+static EVENT_FILTER_CHANGED: "filterChanged"
-format: (date: Date) => string
- parse: (date: string) => Date
-sort: EDateSort
-currentFilter: INumberFilter

+getFormatter(): (date: Date) => string
#createEventList(): string[]
+on(): this
+getValue(): Date[]
+getLabels(): string[]
+getDates(): Date[]
+getDate(): Date
+iterDate(): Date[]
+getSortMethod(): EDateSort
+setSortMethod(): void
+dump(): any
+restore(): void
+toCompareValue(): number[]
+toCompareValueType(): ECompareValueType[]
+isFiltered(): any
+getFilter(): INumberFilter
+setFilter(): void
+filter(): any
+clearFilter(): any

IDatesDesc

+sort?: EDateSort

IDatesColumn

+getDates(): Date[]

IDateDesc

+dateFormat?: string
+dateParse?: string

DatesMapColumn

+static EVENT_SORTMETHOD_CHANGED: "sortMethodChanged"
+static EVENT_FILTER_CHANGED: "filterChanged"
-format: (date: Date) => string
- parse: (date: string) => Date
-sort: EDateSort
-currentFilter: INumberFilter

+getFormatter(): (date: Date) => string
#createEventList(): string[]
+on(): this
- parseValue(): Date
+getDateMap(): { key: string
value: Date
}[]
+iterDate(): Date[]
+getValue(): { key: string
value: Date
}[]
+getLabels(): IKeyValue<string>[]
+getDates(): Date[]
+getDate(): any
+getSortMethod(): EDateSort
+setSortMethod(): any
+dump(): any
+restore(): void
+isFiltered(): any
+getFilter(): INumberFilter
+setFilter(): void
+filter(): any
+clearFilter(): any

GroupColumn

+static EVENT_SORTMETHOD_CHANGED: "sortMethodChanged"
-groupSortMethod: EGroupSortMethod

#createEventList(): string[]
+on(): this
+getLabel(): string
+getValue(): string
+getSortMethod(): EGroupSortMethod
+setSortMethod(): void
+toCompareGroupValue(): string | number
+toCompareGroupValueType(): ECompareValueType.COUNT | ECompareValueType.STRING

HierarchyColumn

+static EVENT_CUTOFF_CHANGED: "cutOffChanged"
+static EVENT_COLOR_MAPPING_CHANGED: "colorMappingChanged"
- hierarchySeparator: string
+hierarchy: Readonly<ICategoryInternalNode>
- currentNode: Readonly<ICategoryInternalNode>
-currentMaxDepth: number
-currentLeaves: Readonly<ICategoryInternalNode>[]
-currentLeavesNameCache: any
-currentLeavesPathCache: any
-colorMapping: ICategoricalColorMappingFunction

- initHierarchy(): ICategoryInternalNode
#createEventList(): string[]
+on(): this
+dump(): any
+restore(): void
+getColorMapping(): ICategoricalColorMappingFunction
+setColorMapping(): void
+getCutOff(): ICutOffNode
+setCutOff(): void
+getCategory(): any
+getValue(): any
+getCategories(): any[]
+iterCategory(): any[]
+getLabel(): any
+getColor(): any
+getLabels(): any
+getValues(): any
+getMap(): any
+getMapLabel(): any
+getSet(): any
+toCompareValue(): any
+toCompareValueType(): any
+group(): IGroup
- updateCaches(): void

ICategoryNode

+children: Readonly<ICategoryNode>[]

ICategoryInternalNode

+path: string
+children: Readonly<ICategoryInternalNode>[]

ICategory

+name: string
+label: string
+color: string
+value: number

IMapColumn<T>

+getMap(): IKeyValue<T>[]
+getMapLabel(): IKeyValue<string>[]

ICategoricalDesc

+categories?: (string | Partial<ICategory>)[]
+categoryOrder?: "given" | "small-to-large" | "large-to-small" | ((categories: readonly ICategory[]) => ICategory[])

ICategoricalFilter

+filter: string | RegExp | string[]
+filterMissing: boolean

ISetCategoricalFilter

+mode?: "every" | "some"

ImpositionBoxPlotColumn

+static EVENT_MAPPING_CHANGED: "mappingChanged"
+static EVENT_COLOR_MAPPING_CHANGED: "colorMappingChanged"

+getLabel(): string
+getColor(): string
#createEventList(): string[]
+on(): this
+getNumberFormat(): any
+getValue(): any
+getNumber(): number
+getRawNumber(): number
+iterNumber(): number[]
+iterRawNumber(): number[]
+getExportValue(): any
+getBoxPlotData(): IBoxPlotData
+getRawBoxPlotData(): IBoxPlotData
+getMapping(): IMappingFunction | ScaleMappingFunction
+getOriginalMapping(): IMappingFunction | ScaleMappingFunction
+getSortMethod(): string
+setSortMethod(): void
+setMapping(): void
+getColorMapping(): IColorMappingFunction
+setColorMapping(): void
+getFilter(): INumberFilter
+setFilter(): void
+getRange(): [string, string]
+toCompareValue(): any
+toCompareValueType(): any
+group(): any
+toCompareGroupValue(): any
+toCompareGroupValueType(): any
+insert(): Column
#insertImpl(): Column
#removeImpl(): boolean

ImpositionCompositeColumn

+static EVENT_MAPPING_CHANGED: "mappingChanged"
+static EVENT_COLOR_MAPPING_CHANGED: "colorMappingChanged"

#createEventList(): string[]
+on(): this
+getLabel(): string
+getColor(): string
+getNumberFormat(): any
+getValue(): any
+getNumber(): number
+getRawNumber(): number
+iterNumber(): number[]
+iterRawNumber(): number[]
+getExportValue(): any
+getMapping(): IMappingFunction | ScaleMappingFunction
+getOriginalMapping(): IMappingFunction | ScaleMappingFunction
+setMapping(): void
+getColorMapping(): IColorMappingFunction
+setColorMapping(): void
+getFilter(): INumberFilter
+setFilter(): void
+getRange(): [string, string]
+toCompareValue(): any
+toCompareValueType(): any
+toCompareGroupValue(): any
+toCompareGroupValueType(): any
+insert(): Column
#insertImpl(): Column
#removeImpl(): boolean

IMapAbleColumn

+getOriginalMapping(): IMappingFunction
+getMapping(): IMappingFunction
+setMapping(): void
+getColorMapping(): IColorMappingFunction
+setColorMapping(): void
+getFilter(): INumberFilter
+setFilter(): void
+getRange(): [string, string]

ImpositionCompositesColumn

+static EVENT_MAPPING_CHANGED: "mappingChanged"
+static EVENT_COLOR_MAPPING_CHANGED: "colorMappingChanged"

+getLabel(): string
+getColor(): string
#createEventList(): string[]
+on(): this
+getNumberFormat(): any
+getValue(): any
+getNumber(): number
+getRawNumber(): number
+getExportValue(): any
+getNumbers(): number[]
+getRawNumbers(): number[]
+iterNumber(): number[]
+iterRawNumber(): number[]
+getBoxPlotData(): IAdvancedBoxPlotData
+getRawBoxPlotData(): IAdvancedBoxPlotData
+getMapping(): IMappingFunction | ScaleMappingFunction
+getOriginalMapping(): IMappingFunction | ScaleMappingFunction
+getSortMethod(): string
+setSortMethod(): void
+setMapping(): void
+getColorMapping(): IColorMappingFunction
+setColorMapping(): void
+getFilter(): INumberFilter
+setFilter(): void
+getRange(): [string, string]
+getMap(): IKeyValue<number>[]
+getMapLabel(): IKeyValue<string>[]
+getLabels(): string[]
+getValues(): number[]
+toCompareValue(): any
+toCompareValueType(): any
+toCompareGroupValue(): any
+toCompareGroupValueType(): any
+insert(): Column
#insertImpl(): Column
#removeImpl(): boolean

INumbersColumn

+getNumbers(): number[]
+getRawNumbers(): number[]

IGroup

+name: string
+color: string
+parent?: Readonly<IGroupParent>

IOrderedGroup

+order: IndicesArray

IGroupParent

+subGroups: (Readonly<IGroupParent> | Readonly<IGroup>)[]

IMultiLevelColumn

+getCollapsed(): boolean
+setCollapsed(): void
+isShowNestedSummaries(): boolean

IMappingFunction

+domain: number[]

+apply(): number
+toJSON(): ITypedDump
+clone(): IMappingFunction
+eq(): boolean
+getRange(): [string, string]

IMapAbleDesc

+map?: ITypedDump
+domain?: [number, number]
+range?: [number, number]
+color?: string
+colorMapping?: string | ITypedDump | ((v: number) => string)

IAdvancedBoxPlotColumn

+getBoxPlotData(): IAdvancedBoxPlotData
+getRawBoxPlotData(): IAdvancedBoxPlotData

LinkColumn

+static EVENT_FILTER_CHANGED: "filterChanged"
+static EVENT_GROUPING_CHANGED: "groupingChanged"
+static EVENT_PATTERN_CHANGED: "patternChanged"
- pattern: string
- patternFunction: (value: string, raw: any) => string
+patternTemplates: string[]
- currentFilter: string | RegExp
-currentGroupCriteria: IStringGroupCriteria
+alignment: EAlignment
+escape: boolean

+setPattern(): void
+getPattern(): string
#createEventList(): string[]
+on(): this
+getValue(): string
+getLink(): ILink
-transformValue(): any
+getLabel(): string
+dump(): any
+restore(): void
+isFiltered(): boolean
+filter(): any
+getFilter(): string | RegExp
+setFilter(): any
+clearFilter(): any
+getGroupCriteria(): IStringGroupCriteria
+setGroupCriteria(): any
+toCompareValue(): any
+toCompareValueType(): any
+toCompareGroupValue(): any
+toCompareGroupValueType(): any
+group(): IGroup

ILinkDesc

+pattern?: string
+patternTemplates?: string[]

IStringDesc

+alignment?: EAlignment
+escape?: boolean

LinkMapColumn

+static EVENT_PATTERN_CHANGED: "patternChanged"
+alignment: EAlignment
+escape: boolean
- pattern: string
- patternFunction: (value: string, raw: any, key: string) => string
+patternTemplates: string[]

+setPattern(): void
+getPattern(): string
#createEventList(): string[]
+on(): this
+getValue(): { key: string
value: string
}[]
+getLabels(): { key: string
value: string
}[]
+getLinkMap(): IKeyValue<ILink>[]
-transformValue(): any
+dump(): any
+restore(): void

LinksColumn

+static EVENT_PATTERN_CHANGED: "patternChanged"
+alignment: EAlignment
+escape: boolean
- pattern: string
- patternFunction: (value: string, raw: any, index: number) => string
+patternTemplates: string[]

+setPattern(): any
+getPattern(): string
#createEventList(): string[]
+on(): this
+getValues(): string[]
+getLabels(): string[]
-transformValue(): any
+getLinks(): ILink[]
+dump(): any
+restore(): void

ScaleMappingFunction

- s: IScale
-type: string

+getRange(): [string, string]
+apply(): number
+invert(): number
+toJSON(): { type: string
domain: number[]
range: number[]
}
+eq(): boolean
+clone(): ScaleMappingFunction

ScriptMappingFunction

-f: IScriptMappingFunctionType
+domain: number[]
+code: string

+getRange(): [string, string]
+apply(): number
+toJSON(): { type: string
code: string
domain: number[]
}
+eq(): boolean
+clone(): ScriptMappingFunction

MultiLevelCompositeColumn

+static EVENT_COLLAPSE_CHANGED: "collapseChanged"
+static EVENT_MULTI_LEVEL_CHANGED: "nestedChildRatio"
+static COLLAPSED_RENDERER: "default"
- adaptChange: (old: number, newValue: number) => void
- collapsed: boolean

#createEventList(): string[]
+on(): this
+setCollapsed(): void
+getCollapsed(): boolean
+isShowNestedSummaries(): boolean
+dump(): any
+restore(): void
+flatten(): any
+insert(): Column
-adaptWidthChange(): void
+removeImpl(): boolean
+setWidth(): void
+getRenderer(): string
+getExportValue(): any

NestedColumn

+toCompareValue(): (string | number)[]
+toCompareValueType(): ECompareValueType[]
+getLabel(): string
+getValue(): string

NumberColumn

+static EVENT_MAPPING_CHANGED: "mappingChanged"
+static EVENT_COLOR_MAPPING_CHANGED: "colorMappingChanged"
+static EVENT_FILTER_CHANGED: "filterChanged"
+static EVENT_SORTMETHOD_CHANGED: "sortMethodChanged"
+static EVENT_GROUPING_CHANGED: "groupingChanged"
- mapping: IMappingFunction
- colorMapping: IColorMappingFunction
- original: IMappingFunction
-deriveMapping: readonly boolean[]
-currentFilter: INumberFilter
-filterAccuracy: number
- numberFormat: (n: number) => string
-currentGroupThresholds: number[]
- groupSortMethod: EAdvancedSortMethod

+getNumberFormat(): (n: number) => string
+onDataUpdate(): void
+dump(): any
+restore(): void
#createEventList(): string[]
+on(): this
+getLabel(): string
+getRange(): [string, string]
+getRawValue(): number
+getExportValue(): any
+getValue(): number
+getNumber(): number
+iterNumber(): number[]
+iterRawNumber(): number[]
+getRawNumber(): number
+toCompareValue(): any
+toCompareValueType(): ECompareValueType
+toCompareGroupValue(): number
+toCompareGroupValueType(): ECompareValueType
+getOriginalMapping(): IMappingFunction
+getMapping(): IMappingFunction
+setMapping(): void
+getColor(): string
+getColorMapping(): IColorMappingFunction
+setColorMapping(): void
+isFiltered(): boolean
+getFilter(): INumberFilter
+setFilter(): void
+filter(): boolean
+clearFilter(): boolean
+getGroupThresholds(): number[]
+setGroupThresholds(): void
+group(): IGroup
+getSortMethod(): EAdvancedSortMethod
+setSortMethod(): void

NumberMapColumn

+static EVENT_MAPPING_CHANGED: "mappingChanged"
+static EVENT_COLOR_MAPPING_CHANGED: "colorMappingChanged"
+static EVENT_SORTMETHOD_CHANGED: "sortMethodChanged"
+static EVENT_FILTER_CHANGED: "filterChanged"
- numberFormat: (n: number) => string
- sort: EAdvancedSortMethod
- mapping: IMappingFunction
- original: IMappingFunction
- colorMapping: IColorMappingFunction
- currentFilter: INumberFilter

+getNumberFormat(): (n: number) => string
+toCompareValue(): number
+toCompareValueType(): ECompareValueType
+getBoxPlotData(): IAdvancedBoxPlotData
+getRange(): [string, string]
+getRawBoxPlotData(): IAdvancedBoxPlotData
+getNumber(): number
+getRawNumber(): number
+iterNumber(): number[]
+iterRawNumber(): number[]
+getValue(): { key: string
value: number
}[]
+getRawValue(): IKeyValue<number>[]
+getExportValue(): any
+getLabels(): { key: string
value: string
}[]
+getSortMethod(): EAdvancedSortMethod
+setSortMethod(): void
+dump(): any
+restore(): void
#createEventList(): string[]
+on(): this
+getOriginalMapping(): IMappingFunction
+getMapping(): IMappingFunction
+setMapping(): void
+getColor(): any
+getColorMapping(): IColorMappingFunction
+setColorMapping(): void
+isFiltered(): any
+getFilter(): INumberFilter
+setFilter(): void
+filter(): any
+clearFilter(): any

INumberMapDesc

+sort?: EAdvancedSortMethod

NumbersColumn

+static EVENT_MAPPING_CHANGED: "mappingChanged"
+static EVENT_COLOR_MAPPING_CHANGED: "colorMappingChanged"
+static EVENT_SORTMETHOD_CHANGED: "sortMethodChanged"
+static EVENT_FILTER_CHANGED: "filterChanged"
+static CENTER: 0
- numberFormat: (n: number) => string
- sort: EAdvancedSortMethod
- mapping: IMappingFunction
- original: IMappingFunction
- colorMapping: IColorMappingFunction
- deriveMapping: readonly boolean[]
- currentFilter: INumberFilter

+onDataUpdate(): void
+getNumberFormat(): (n: number) => string
+toCompareValue(): number
+toCompareValueType(): ECompareValueType
+getRawNumbers(): number[]
+getBoxPlotData(): IAdvancedBoxPlotData
+getRange(): [string, string]
+getRawBoxPlotData(): IAdvancedBoxPlotData
+getNumbers(): number[]
+getNumber(): number
+getRawNumber(): number
+getValue(): any
+getValues(): number[]
+iterNumber(): number[]
+iterRawNumber(): number[]
+getRawValue(): number[]
+getExportValue(): any
+getLabels(): string[]
+getSortMethod(): EAdvancedSortMethod
+setSortMethod(): void
+dump(): any
+restore(): void
#createEventList(): string[]
+on(): this
+getOriginalMapping(): IMappingFunction
+getMapping(): IMappingFunction
+setMapping(): void
+getColor(): any
+getColorMapping(): IColorMappingFunction
+setColorMapping(): void
+isFiltered(): any
+getFilter(): INumberFilter
+setFilter(): void
+filter(): any
+clearFilter(): any

INumbersDesc

+sort?: EAdvancedSortMethod

OrdinalColumn

+static EVENT_MAPPING_CHANGED: "mappingChanged"
+static EVENT_FILTER_CHANGED: "filterChanged"
+static EVENT_COLOR_MAPPING_CHANGED: "colorMappingChanged"
+categories: ICategory[]
- colorMapping: ICategoricalColorMappingFunction
- lookup: any
-currentFilter: ICategoricalFilter

+onDataUpdate(): void
#createEventList(): string[]
+on(): this
+getNumberFormat(): any
+getValue(): any
+getCategory(): any
+getCategories(): any[]
+iterCategory(): any[]
+iterNumber(): any[]
+iterRawNumber(): any[]
+getColor(): any
+getLabel(): any
+getLabels(): any
+getValues(): any
+getMap(): any
+getMapLabel(): any
+getSet(): any
+getNumber(): any
+getRawNumber(): any
+getExportValue(): any
+dump(): any
+restore(): void
+getMapping(): number[]
+setMapping(): void
+getColorMapping(): ICategoricalColorMappingFunction
+setColorMapping(): any
+isFiltered(): boolean
+filter(): boolean
+group(): any
+getFilter(): ICategoricalFilter
+setFilter(): any
+clearFilter(): any
+toCompareValue(): any
+toCompareValueType(): any
+getRenderer(): string

RankColumn

+getLabel(): string
+getRaw(): number
+getValue(): number

Ranking

+static EVENT_WIDTH_CHANGED: "widthChanged"
+static EVENT_FILTER_CHANGED: "filterChanged"
+static EVENT_LABEL_CHANGED: "labelChanged"
+static EVENT_ADD_COLUMN: "addColumn"
+static EVENT_MOVE_COLUMN: "moveColumn"
+static EVENT_REMOVE_COLUMN: "removeColumn"
+static EVENT_DIRTY: "dirty"
+static EVENT_DIRTY_HEADER: "dirtyHeader"
+static EVENT_DIRTY_VALUES: "dirtyValues"
+static EVENT_DIRTY_CACHES: "dirtyCaches"
+static EVENT_COLUMN_VISIBILITY_CHANGED: "visibilityChanged"
+static EVENT_SORT_CRITERIA_CHANGED: "sortCriteriaChanged"
+static EVENT_GROUP_CRITERIA_CHANGED: "groupCriteriaChanged"
+static EVENT_GROUP_SORT_CRITERIA_CHANGED: "groupSortCriteriaChanged"
+static EVENT_DIRTY_ORDER: "dirtyOrder"
+static EVENT_ORDER_CHANGED: "orderChanged"
+static EVENT_GROUPS_CHANGED: "groupsChanged"
-static FORWARD_COLUMN_EVENTS: string[]
-static COLUMN_GROUP_SORT_DIRTY: string[]
-static COLUMN_SORT_DIRTY: string[]
-static COLUMN_GROUP_DIRTY: string[]
- label: string
- sortCriteria: ISortCriteria[]
-groupColumns: Column[]
-groupSortCriteria: ISortCriteria[]
-columns: Column[]
+dirtyOrder: (reason?: EDirtyReason[]) => void
-dirtyOrderSortDirty: () => void
-dirtyOrderGroupDirty: () => void
- dirtyOrderGroupSortDirty: () => void
- dirtyOrderFiltering: () => void
-groups: IOrderedGroup[]
- order: IndicesArray
- index2pos: IndicesArray
+id: string

#createEventList(): string[]
+on(): this
+assignNewId(): void
+getLabel(): string
+setLabel(): void
+setGroups(): void
- unifyGroups(): void
+getRank(): number
+getOrder(): IndicesArray
+getOrderLength(): number
+getGroups(): IOrderedGroup[]
+getFlatGroups(): Readonly<IGroup | IGroupParent>[]
+dump(): IRankingDump
+restore(): void
+flatten(): number
+getPrimarySortCriteria(): ISortCriteria
+getSortCriteria(): ISortCriteria[]
+getGroupSortCriteria(): ISortCriteria[]
+toggleSorting(): boolean
-toggleSortingLogic(): ISortCriteria[]
+toggleGrouping(): boolean
+getGroupCriteria(): Column[]
+sortBy(): boolean
+groupSortBy(): boolean
- hierarchyLogic(): T[]
+groupBy(): boolean
+setSortCriteria(): boolean
+setGroupCriteria(): boolean
- autoAdaptAggregationColumn(): void
+toggleGroupSorting(): boolean
+setGroupSortCriteria(): boolean
-triggerGroupResort(): void
-triggerResort(): void
+insert(): Column
+move(): Column
+moveAfter(): Column
+moveBefore(): Column
+findByPath(): Column
+indexOf(): number
+at(): Column
+insertAfter(): Column
+insertBefore(): Column
+push(): Column
+remove(): boolean
+clear(): void
+find(): Column
+isFiltered(): boolean
+filter(): boolean
+clearFilters(): boolean
+findMyRanker(): this
+markDirty(): void

ReduceColumn

+static EVENT_REDUCE_CHANGED: "reduceChanged"
- reduce: EAdvancedSortMethod

+getColor(): any
#compute(): any
#createEventList(): string[]
+on(): this
+getReduce(): EAdvancedSortMethod
+setReduce(): void
+dump(): any
+restore(): void
+getExportValue(): any

ColumnWrapper

- c: Column
+v: any
+raw: number

LazyColumnWrapper

- c: Column
- row: IDataRow

ScriptColumn

+static EVENT_MAPPING_CHANGED: "mappingChanged"
+static EVENT_COLOR_MAPPING_CHANGED: "colorMappingChanged"
+static EVENT_SCRIPT_CHANGED: "scriptChanged"
+static DEFAULT_SCRIPT: "let s = 0
col.forEach((c) => s += c.v)
return s / col.length"
-script: string
-f: (children: any, values: number[], raws: any[], col: ColumnContext, row: any, index: number) => number
- mapping: IMappingFunction
- original: IMappingFunction
-colorMapping: IColorMappingFunction
- currentFilter: INumberFilter

#createEventList(): string[]
+on(): this
+setScript(): void
+getScript(): string
+dump(): any
+restore(): void
#compute(): any
+getExportValue(): any
+getRange(): [string, string]
+getOriginalMapping(): IMappingFunction
+getMapping(): IMappingFunction
+setMapping(): void
+getColor(): any
+getColorMapping(): IColorMappingFunction
+setColorMapping(): void
+isFiltered(): any
+getFilter(): INumberFilter
+setFilter(): void
+filter(): any
+clearFilter(): any

IColumnWrapper

+v: any
+raw: number
+type: string
+name: string
+id: string

IScriptDesc

+script?: string

SelectionColumn

+static EVENT_FILTER_CHANGED: "filterChanged"
+static EVENT_SELECT: "select"
-static SELECTED_GROUP: IGroup
-static NOT_SELECTED_GROUP: IGroup
-currentFilter: any

#createEventList(): string[]
+on(): this
+setValue(): boolean
+setValues(): boolean
-setImpl(): boolean
+toggleValue(): boolean
+toCompareValue(): 0 | 1
+toCompareValueType(): ECompareValueType
+group(): any
+dump(): any
+restore(): void
+isFiltered(): boolean
+filter(): any
+getFilter(): number[]
+setFilter(): void
+clearFilter(): boolean

ISelectionColumnDesc

+setter(): void
+setterAll(): void

SetColumn

+static EVENT_FILTER_CHANGED: "filterChanged"
+static EVENT_COLOR_MAPPING_CHANGED: "colorMappingChanged"
+categories: ICategory[]
- separator: RegExp
- lookup: any
- colorMapping: ICategoricalColorMappingFunction
- currentFilter: ISetCategoricalFilter

#createEventList(): string[]
+on(): this
+getValue(): string[]
+getLabel(): string
- normalize(): any
+getSet(): any
+getSortedSet(): any
+getCategories(): any
+getColors(): any
+getColorMapping(): ICategoricalColorMappingFunction
+setColorMapping(): any
+getValues(): any[]
+getLabels(): string[]
+getMap(): any
+getMapLabel(): any
+iterCategory(): any
+dump(): any
+restore(): void
+isFiltered(): boolean
+filter(): boolean
+getFilter(): ISetCategoricalFilter
+setFilter(): void
+clearFilter(): any
+toCompareValue(): any[]
+toCompareValueType(): ECompareValueType[]
+group(): IGroup

ISetDesc

+separator?: string

StackColumn

+static EVENT_COLLAPSE_CHANGED: "collapseChanged"
+static EVENT_WEIGHTS_CHANGED: "weightsChanged"
+static EVENT_MULTI_LEVEL_CHANGED: "nestedChildRatio"
+static COLLAPSED_RENDERER: "number"
- adaptChange: (old: number, newValue: number) => void
-collapsed: boolean

#createEventList(): string[]
+on(): this
+setCollapsed(): void
+getCollapsed(): boolean
+isShowNestedSummaries(): boolean
+flatten(): number
+dump(): any
+restore(): void
+insert(): Column
+push(): Column
+insertAfter(): Column
-adaptWidthChange(): void
+getWeights(): number[]
+setWeights(): void
+removeImpl(): boolean
+setWidth(): void
#compute(): number
+getRenderer(): string
+getExportValue(): any

StringMapColumn

+alignment: EAlignment
+escape: boolean

+on(): this
+getValue(): { key: string
value: string
}[]
+getMapValue(): { key: string
value: string
}[]

StringsColumn

+alignment: EAlignment
+escape: boolean

+getValues(): string[]

ACommonDataProvider

- rankingIndex: number
- columns: IColumnDesc[]

+cloneRanking(): Ranking
+pushDesc(): void
+clearColumns(): void
+removeDesc(): boolean
+getColumns(): IColumnDesc[]
+findDesc(): IColumnDesc
+toDescRef(): any
+fromDescRef(): any
+restore(): void
+nextRankingId(): string

ADataProvider

+static EVENT_SELECTION_CHANGED: "selectionChanged"
+static EVENT_DATA_CHANGED: "dataChanged"
+static EVENT_ADD_COLUMN: "addColumn"
+static EVENT_MOVE_COLUMN: "moveColumn"
+static EVENT_REMOVE_COLUMN: "removeColumn"
+static EVENT_ADD_RANKING: "addRanking"
+static EVENT_REMOVE_RANKING: "removeRanking"
+static EVENT_DIRTY: "dirty"
+static EVENT_DIRTY_HEADER: "dirtyHeader"
+static EVENT_DIRTY_VALUES: "dirtyValues"
+static EVENT_DIRTY_CACHES: "dirtyCaches"
+static EVENT_ORDER_CHANGED: "orderChanged"
+static EVENT_SHOWTOPN_CHANGED: "showTopNChanged"
+static EVENT_ADD_DESC: "addDesc"
+static EVENT_CLEAR_DESC: "clearDesc"
+static EVENT_REMOVE_DESC: "removeDesc"
+static EVENT_JUMP_TO_NEAREST: "jumpToNearest"
+static EVENT_GROUP_AGGREGATION_CHANGED: "aggregate"
+static EVENT_BUSY: "busy"
-static FORWARD_RANKING_EVENTS: string[]
- rankings: Ranking[]
- selection: OrderedSet<number>
- aggregations: any
- uid: number
-typeFactory: ITypeFactory
- options: Readonly<IDataProviderOptions>
+columnTypes: { [columnType: string]: IColumnConstructor
}
+colorMappingFunctionTypes: { [colorMappingFunctionType: string]: IColorMappingFunctionConstructor
}
+mappingFunctionTypes: { [mappingFunctionType: string]: IMappingFunctionConstructor
}
-showTopN: number

-createTypeFactory(): ITypeFactory
+getTypeFactory(): ITypeFactory
#createEventList(): string[]
+on(): this
+abstract getTotalNumberOfRows(): number
+abstract getColumns(): IColumnDesc[]
+abstract getTaskExecutor(): IRenderTasks
+pushRanking(): Ranking
#fireBusy(): void
+takeSnapshot(): Ranking
+insertRanking(): void
-triggerReorder(): void
+removeRanking(): boolean
+clearRankings(): void
+clearFilters(): void
+getRankings(): Ranking[]
+getFirstRanking(): Ranking
+getLastRanking(): Ranking
+ensureOneRanking(): void
+destroy(): void
+cleanUpRanking(): void
+abstract cloneRanking(): Ranking
+push(): Column
+insert(): Column
- nextId(): string
-fixDesc(): IColumnDesc
#cleanDesc(): IColumnDesc
+create(): Column
#patchColumn(): Column
#instantiateColumn(): Column
+clone(): Column
+restoreColumn(): Column
+find(): Column
+dump(): IDataProviderDump
+dumpColumn(): IColumnDump
+toDescRef(): any
+fromDescRef(): any
+restoreRanking(): Ranking
+restore(): void
+abstract findDesc(): IColumnDesc
+deriveDefault(): Ranking
+isAggregated(): boolean
+getAggregationState(): EAggregationState
+setAggregated(): void
+setAggregationState(): void
+getTopNAggregated(): any
- unaggregateParents(): boolean
+getAggregationStrategy(): IAggregationStrategy
- initAggregateState(): void
+setTopNAggregated(): void
+aggregateAllOf(): void
+getShowTopN(): number
+setShowTopN(): void
+abstract sort(): Promise<{ groups: IOrderedGroup[]
index2pos: IndicesArray
}> | { groups: IOrderedGroup[]
index2pos: IndicesArray
}
+abstract view(): any[] | Promise<any[]>
+abstract getRow(): IDataRow | Promise<IDataRow>
+abstract mappingSample(): ISequence<number> | Promise<ISequence<number>>
+isSelected(): any
+select(): void
+abstract searchAndJump(): void | number[]
+jumpToNearest(): void
+selectAll(): void
+selectAllOf(): void
+setSelection(): void
+toggleSelection(): boolean
+deselect(): void
+deselectAll(): void
+selectedRows(): any[] | Promise<any[]>
+getSelection(): any
+clearSelection(): void
+exportTable(): string | Promise<string>
+exportSelection(): string | Promise<string>

IDataProvider

+columnTypes: { [columnType: string]: IColumnConstructor
}

+getTotalNumberOfRows(): number
+getTaskExecutor(): IRenderTasks
+getTypeFactory(): ITypeFactory
+takeSnapshot(): void
+selectAllOf(): void
+getSelection(): number[]
+setSelection(): void
+toggleSelection(): boolean
+isSelected(): boolean
+removeRanking(): void
+ensureOneRanking(): void
+find(): Column
+clone(): Column
+create(): Column
+toDescRef(): any
+fromDescRef(): IColumnDesc
+mappingSample(): ISequence<number> | Promise<ISequence<number>>
+searchAndJump(): void | number[]
+jumpToNearest(): any
+getRankings(): Ranking[]
+getFirstRanking(): Ranking
+getLastRanking(): Ranking
+getColumns(): IColumnDesc[]
+getAggregationStrategy(): IAggregationStrategy
+isAggregated(): boolean
+setAggregationState(): void
+getAggregationState(): EAggregationState
+aggregateAllOf(): void
+getTopNAggregated(): number
+setTopNAggregated(): void
+setShowTopN(): void
+getShowTopN(): number
+getRow(): IDataRow | Promise<IDataRow>
+clearFilters(): void

DirectRenderTasks

#cache: any

+setData(): void
+dirtyColumn(): void
+dirtyRanking(): void
+preCompute(): void
+preComputeData(): void
+preComputeCol(): void
+copyData2Summary(): void
+copyCache(): void
+sort(): any
+groupCompare(): IRenderTask<ICompareValue[]>
+groupRows(): IRenderTask<T>
+groupExampleRows(): IRenderTask<T>
+groupBoxPlotStats(): TaskNow<{ group: any
summary: IAdvancedBoxPlotData
data: IAdvancedBoxPlotData
}>
+groupNumberStats(): TaskNow<{ group: any
summary: IStatistics
data: IStatistics
}>
+groupCategoricalStats(): TaskNow<{ group: any
summary: ICategoricalStatistics
data: ICategoricalStatistics
}>
+groupDateStats(): TaskNow<{ group: any
summary: IDateStatistics
data: IDateStatistics
}>
+groupStringStats(): TaskNow<{ group: any
summary: IStringStatistics
data: IStringStatistics
}>
+summaryBoxPlotStats(): TaskNow<{ summary: IAdvancedBoxPlotData
data: IAdvancedBoxPlotData
}>
+summaryNumberStats(): TaskNow<{ summary: IStatistics
data: IStatistics
}>
+summaryCategoricalStats(): TaskNow<{ summary: ICategoricalStatistics
data: ICategoricalStatistics
}>
+summaryDateStats(): TaskNow<{ summary: IDateStatistics
data: IDateStatistics
}>
+summaryStringStats(): TaskNow<{ summary: IStringStatistics
data: IStringStatistics
}>
-summaryNumberStatsD(): { summary: IStatistics
data: IStatistics
}
- summaryBoxPlotStatsD(): { summary: IAdvancedBoxPlotData
data: IAdvancedBoxPlotData
}
-summaryCategoricalStatsD(): { summary: ICategoricalStatistics
data: ICategoricalStatistics
}
-summaryDateStatsD(): { summary: IDateStatistics
data: IDateStatistics
}
-summaryStringStatsD(): { summary: IStringStatistics
data: IStringStatistics
}
-cached(): T
+dataBoxPlotStats(): IAdvancedBoxPlotData
+dataNumberStats(): IStatistics
+dataCategoricalStats(): ICategoricalStatistics
+dataDateStats(): IDateStatistics
+dataStringStats(): IStringStatistics
+terminate(): void

ARenderTasks

#valueCacheData: any
#byIndex: (i: number) => IDataRow
#options: ARenderTaskOptions
#data: IDataRow[]

#byOrder(): ISequence<IDataRow>
#byOrderAcc(): ISequence<T>
- builder(): Iterator<R>
- builderForEach(): Iterator<R>
#boxplotBuilder(): any
#resolveDomain(): [number, number]
#statsBuilder(): any
- numberStatsBuilder(): any
#dateStatsBuilder(): any
#categoricalStatsBuilder(): any
#stringStatsBuilder(): Iterator<R>
+dirtyColumn(): void
#setValueCacheData(): void
+valueCache(): (dataIndex: number) => any

IRenderTaskExecutor

+setData(): void
+dirtyColumn(): void
+dirtyRanking(): void
+groupCompare(): IRenderTask<ICompareValue[]>
+preCompute(): void
+preComputeCol(): void
+preComputeData(): void
+copyData2Summary(): void
+copyCache(): void
+sort(): Promise<IndicesArray>
+terminate(): void
+valueCache(): (dataIndex: number) => any

LocalDataProvider

- ooptions: ILocalDataProviderOptions
- reorderAll: () => void
-_dataRows: IDataRow[]
-filter: (row: IDataRow) => boolean
-tasks: IRenderTaskExecutor
- mapToDataRow: (i: number) => IDataRow
-_data: any[]

+setFilter(): void
+getFilter(): (row: IDataRow) => boolean
+getTotalNumberOfRows(): number
+getTaskExecutor(): IRenderTaskExecutor
+destroy(): void
+setData(): void
-dataChanged(): void
+clearData(): void
+appendData(): void
+cloneRanking(): Ranking
-trackRanking(): void
#patchColumn(): Column
+cleanUpRanking(): void
- resolveFilter(): (Column | ((d: IDataRow) => boolean))[]
- noSorting(): { groups: any[]
index2pos: Uint8Array
}
- createSorter(): { maxDataIndex: number
lookups: CompareLookup
groupOrder: ISortHelper[]
}
-sortGroup(): Promise<IOrderedGroup>
-sortGroups(): IOrderedGroup[]
- index2pos(): { groups: IOrderedGroup[]
index2pos: Uint8Array
}
+sort(): any
+viewRaw(): any[]
+viewRawRows(): IDataRow[]
+getRow(): IDataRow
+seq(): ISequence<IDataRow>
+view(): any[]
+mappingSample(): ISequence<number>
+searchAndJump(): number[]
+search(): number[]

ILocalDataProviderOptions

+filterGlobally: boolean
+jumpToSearchResult: boolean
+taskExecutor: "direct" | "scheduled"

ARenderTaskOptions

+stringTopNCount: number | readonly string[]

RemoteDataProvider

- ooptions: IRemoteDataProviderOptions
-cache: any
-server: IServerData

+getTotalNumberOfRows(): any
+getTaskExecutor(): IRenderTasks
+sort(): Promise<{ groups: IOrderedGroup[]
index2pos: IndicesArray
}> | { groups: IOrderedGroup[]
index2pos: IndicesArray
}
- loadFromServer(): Promise<{ v: any
dataIndex: number
}[]>
+view(): Promise<any[]>
-computeMissing(): number[]
- loadInCache(): void
+fetch(): Promise<IDataRow>[][]
+getRow(): any
+mappingSample(): Promise<number[]>
+searchAndJump(): void

ScheduleRenderTasks

-cache: any
-tasks: TaskScheduler
-workers: WorkerTaskScheduler

+setData(): void
+dirtyColumn(): void
+dirtyRanking(): void
+preCompute(): void
+preComputeData(): void
+preComputeCol(): void
+copyData2Summary(): void
+copyCache(): void
+groupCompare(): TaskLater<unknown>
+groupRows(): IRenderTask<unknown>
+groupExampleRows(): TaskNow<T>
+groupBoxPlotStats(): IRenderTask<unknown>
+groupNumberStats(): IRenderTask<unknown>
+groupCategoricalStats(): IRenderTask<unknown>
+groupDateStats(): IRenderTask<unknown>
+groupStringStats(): IRenderTask<unknown>
+summaryBoxPlotStats(): IRenderTask<unknown>
+summaryNumberStats(): IRenderTask<unknown>
+summaryCategoricalStats(): IRenderTask<unknown>
+summaryDateStats(): IRenderTask<unknown>
+summaryStringStats(): IRenderTask<unknown>
- cached(): IRenderTask<T>
- chain(): IRenderTask<U>
- isValidCacheEntry(): boolean
-chainCopy(): IRenderTask<U>
+dataBoxPlotStats(): IRenderTask<unknown>
+dataNumberStats(): IRenderTask<unknown>
+dataCategoricalStats(): IRenderTask<unknown>
+dataStringStats(): IRenderTask<unknown>
+dataDateStats(): IRenderTask<unknown>
+sort(): any
+terminate(): void

TaskNow<T>

+v: T

+then(): U

TaskLater<T>

+v: IAbortAblePromise<T>

+then(): IAbortAblePromise<U>

IRenderTask<T>

+then(): any

IRenderTasks

+groupRows(): IRenderTask<T>
+groupExampleRows(): IRenderTask<T>
+groupBoxPlotStats(): IRenderTask<{ group: IAdvancedBoxPlotData
summary: IAdvancedBoxPlotData
data: IAdvancedBoxPlotData
}>
+groupNumberStats(): IRenderTask<{ group: IStatistics
summary: IStatistics
data: IStatistics
}>
+groupCategoricalStats(): IRenderTask<{ group: ICategoricalStatistics
summary: ICategoricalStatistics
data: ICategoricalStatistics
}>
+groupDateStats(): IRenderTask<{ group: IDateStatistics
summary: IDateStatistics
data: IDateStatistics
}>
+groupStringStats(): IRenderTask<{ group: IStringStatistics
summary: IStringStatistics
data: IStringStatistics
}>
+summaryBoxPlotStats(): IRenderTask<{ summary: IAdvancedBoxPlotData
data: IAdvancedBoxPlotData
}>
+summaryNumberStats(): IRenderTask<{ summary: IStatistics
data: IStatistics
}>
+summaryCategoricalStats(): IRenderTask<{ summary: ICategoricalStatistics
data: ICategoricalStatistics
}>
+summaryDateStats(): IRenderTask<{ summary: IDateStatistics
data: IDateStatistics
}>
+summaryStringStats(): IRenderTask<{ summary: IStringStatistics
data: IStringStatistics
}>

AAggregatedGroupRenderer<T>

+abstract title: string

+abstract canRender(): boolean
+abstract create(): ICellRenderer
#abstract aggregatedIndex(): { row: IDataRow
index: number
}
+createGroup(): IGroupCellRenderer
+createSummary(): ISummaryRenderer

ICellRendererFactory

+title: string
+groupTitle?: string
+summaryTitle?: string

+canRender(): boolean
+create(): ICellRenderer
+createGroup(): IGroupCellRenderer
+createSummary(): ISummaryRenderer

ActionRenderer

+title: "Default"

+canRender(): boolean
+create(): ICellRenderer
+createGroup(): IGroupCellRenderer
+createSummary(): ISummaryRenderer

AggregateGroupRenderer

+title: string

+canRender(): boolean
+create(): ICellRenderer
+createGroup(): IGroupCellRenderer
+createSummary(): ISummaryRenderer

AnnotationRenderer

+title: string

+canRender(): boolean
+create(): ICellRenderer

StringCellRenderer

+title: string

+canRender(): boolean
+create(): ICellRenderer
+createGroup(): IGroupCellRenderer
-static interactiveSummary(): (actCol: StringColumn) => void
+createSummary(): ISummaryRenderer

ANumbersCellRenderer

+abstract title: string

#abstract createContext(): { clazz: string
templateRow: string
update: (row: HTMLElement, data: number[], raw: number[], d: IDataRow, tooltipPrefix?: string) => void
render: (ctx: CanvasRenderingContext2D, data: number[], d: IDataRow) => void
}
+static choose(): { normalized: number[]
raw: number[]
row: IDataRow
}
+create(): ICellRenderer
+createGroup(): IGroupCellRenderer

BarCellRenderer

+title: string
- renderValue: boolean

+canRender(): boolean
+create(): ICellRenderer
+createGroup(): IGroupCellRenderer
+createSummary(): ISummaryRenderer

BooleanCellRenderer

+title: string

+canRender(): boolean
+create(): ICellRenderer

DefaultCellRenderer

+title: string
+groupTitle: string
+summaryTitle: string

+canRender(): boolean
+create(): ICellRenderer
+createGroup(): IGroupCellRenderer
+createSummary(): ISummaryRenderer

BoxplotCellRenderer

+title: string

+canRender(): boolean
+create(): ICellRenderer
+createGroup(): IGroupCellRenderer
+createSummary(): ISummaryRenderer

BrightnessCellRenderer

+title: string

+canRender(): boolean
+create(): ICellRenderer
+createGroup(): IGroupCellRenderer
+createSummary(): ISummaryRenderer

CategoricalCellRenderer

+title: string
+groupTitle: string

+canRender(): boolean
+create(): ICellRenderer
+createGroup(): IGroupCellRenderer
+createSummary(): ISummaryRenderer

CategoricalHeatmapCellRenderer

+title: string

+canRender(): boolean
-createContext(): { template: string
render: (ctx: CanvasRenderingContext2D, data: ICategory[], height: number) => void
width: number
mover: (n: HTMLElement, values: string[]) => (evt: MouseEvent) => void
}
+create(): ICellRenderer
+createGroup(): IGroupCellRenderer
+createSummary(): ISummaryRenderer

CategoricalsHistogramCellRenderer

+title: string

+canRender(): boolean
+create(): ICellRenderer

CategoricalStackedDistributionlCellRenderer

+title: string

+canRender(): boolean
+create(): ICellRenderer
+createGroup(): IGroupCellRenderer
+createSummary(): ISummaryRenderer

CategoricalTickCellRenderer

+title: string
- renderValue: boolean

+canRender(): boolean
+create(): ICellRenderer
+createGroup(): IGroupCellRenderer
+createSummary(): ISummaryRenderer

CircleCellRenderer

+title: string
- renderValue: boolean

+canRender(): boolean
+create(): ICellRenderer
+createGroup(): IGroupCellRenderer
+createSummary(): ISummaryRenderer

DateCellRenderer

+title: string
+groupTitle: string
+summaryTitle: string

+canRender(): boolean
+create(): ICellRenderer
+createGroup(): IGroupCellRenderer
+createSummary(): ISummaryRenderer

DateHistogramCellRenderer

+title: string

+canRender(): boolean
+create(): ICellRenderer
+createGroup(): IGroupCellRenderer
+createSummary(): ISummaryRenderer

DotCellRenderer

+title: string
+groupTitle: string
- renderValue: boolean

+canRender(): boolean
- static getCanvasRenderer(): { template: string
render: (ctx: CanvasRenderingContext2D, vs: { value: number
color: string
}[], width: number) => void
width: number
}
- static getDOMRenderer(): { template: string
update: (n: HTMLElement, data: ISequence<{ value: number
label: string
color: string
}>) => void
render: (ctx: CanvasRenderingContext2D, vs: number[], colors: string[], width: number) => void
}
- static getSingleDOMRenderer(): { template: string
update: (n: HTMLElement, value: number, label: string, color: string) => void
render: (ctx: CanvasRenderingContext2D, value: number, color: string, width: number) => void
}
+create(): ICellRenderer
+createGroup(): IGroupCellRenderer
+createSummary(): ISummaryRenderer

GroupCellRenderer

+title: string

+canRender(): boolean
+create(): ICellRenderer
+createGroup(): IGroupCellRenderer
+createSummary(): ISummaryRenderer

HeatmapCellRenderer

+title: string

+canRender(): boolean
-createContext(): { template: string
render: (ctx: CanvasRenderingContext2D, data: number[], item: IDataRow, height: number) => void
width: number
mover: (n: HTMLElement, values: string[], prefix?: string) => (evt: MouseEvent) => void
}
+create(): ICellRenderer
+createGroup(): IGroupCellRenderer
+createSummary(): ISummaryRenderer

HistogramCellRenderer

+title: string

+canRender(): boolean
+create(): ICellRenderer
+createGroup(): IGroupCellRenderer
+createSummary(): ISummaryRenderer

ImageCellRenderer

+title: string

+canRender(): boolean
+create(): ICellRenderer
+createGroup(): IGroupCellRenderer
+createSummary(): ISummaryRenderer

IRenderContext

+tasks: IRenderTasks
+idPrefix: string
+provider: IDataProvider
+dialogManager: DialogManager

+renderer(): ICellRenderer
+groupRenderer(): IGroupCellRenderer
+summaryRenderer(): ISummaryRenderer
+asElement(): HTMLElement
+sanitize(): string
+colWidth(): number

InterleavingCellRenderer

+title: string

+canRender(): boolean
+create(): ICellRenderer
+createGroup(): IGroupCellRenderer
+createSummary(): ISummaryRenderer

LinkCellRenderer

+title: string

+canRender(): boolean
+create(): ICellRenderer
-static exampleText(): [ILink[], boolean]
+createGroup(): IGroupCellRenderer
+createSummary(): ISummaryRenderer

LinkMapCellRenderer

+title: string

+canRender(): boolean
+create(): ICellRenderer
-static example(): [ILink[], boolean]
+createGroup(): IGroupCellRenderer
+createSummary(): ISummaryRenderer

LoadingCellRenderer

+title: string

+canRender(): boolean
+create(): { template: string
update: () => void
}
+createGroup(): IGroupCellRenderer
+createSummary(): ISummaryRenderer

MapBarCellRenderer

+title: string

+canRender(): boolean
+create(): ICellRenderer
+createGroup(): IGroupCellRenderer
+createSummary(): ISummaryRenderer

MultiLevelCellRenderer

+title: string
-stacked: boolean

+canRender(): boolean
+create(): ICellRenderer
+createGroup(): IGroupCellRenderer
#aggregatedIndex(): { index: any
row: any
}

RankCellRenderer

+title: string

+canRender(): boolean
+create(): ICellRenderer
+createGroup(): IGroupCellRenderer
+createSummary(): ISummaryRenderer

SelectionRenderer

+title: string

+canRender(): boolean
+create(): ICellRenderer
+createGroup(): IGroupCellRenderer
+createSummary(): ISummaryRenderer

SetCellRenderer

+title: string

+canRender(): boolean
-static createDOMContext(): { templateRow: string
render: (n: HTMLElement, value: (number | boolean)[]) => void
}
+create(): ICellRenderer
+createGroup(): IGroupCellRenderer
+createSummary(): ISummaryRenderer

SparklineCellRenderer

+title: string

+canRender(): boolean
+create(): ICellRenderer
+createGroup(): IGroupCellRenderer
+createSummary(): ISummaryRenderer

StringHistogramCellRenderer

+title: string

+createGroup(): IGroupCellRenderer
+createSummary(): ISummaryRenderer

TableCellRenderer

+title: string

+canRender(): boolean
+create(): ICellRenderer
-static template(): string
-createFixed(): { template: string
update: (node: HTMLElement, d: IDataRow) => void
}
+createGroup(): IGroupCellRenderer
- createFixedGroup(): { template: string
update: (node: HTMLElement, group: IOrderedGroup) => any
}
+createSummary(): ISummaryRenderer

TickCellRenderer

+title: string
+groupTitle: string
- renderValue: boolean

+canRender(): boolean
+create(): ICellRenderer
+createGroup(): IGroupCellRenderer
+createSummary(): ISummaryRenderer

UpSetCellRenderer

+title: string

+canRender(): boolean
-static calculateSetPath(): { left: number
right: number
}
-static createDOMContext(): { template: string
render: (n: HTMLElement, value: boolean[]) => void
}
+create(): ICellRenderer
+createGroup(): IGroupCellRenderer
+createSummary(): ISummaryRenderer

VerticalBarCellRenderer

+title: string

+canRender(): boolean
-static compute(): { height: number
bottom: number
}
#createContext(): { clazz: any
templateRow: string
update: (row: HTMLElement, data: number[], raw: number[], item: IDataRow, tooltipPrefix?: string) => void
render: (ctx: CanvasRenderingContext2D, data: number[], item: IDataRow) => void
}
+createSummary(): ISummaryRenderer

ViolinPlotCellRenderer

+title: string

+canRender(): boolean
+create(): ICellRenderer
+createGroup(): IGroupCellRenderer
+createSummary(): ISummaryRenderer

ALineUp

+static EVENT_SELECTION_CHANGED: "selectionChanged"
+static EVENT_DIALOG_OPENED: "dialogOpened"
+static EVENT_DIALOG_CLOSED: "dialogClosed"
+static EVENT_HIGHLIGHT_CHANGED: "highlightChanged"
- highlightListeners: number
+isBrowserSupported: boolean
+node: HTMLElement
-_data: ADataProvider

#createEventList(): string[]
+on(): this
+destroy(): void
+dump(): IDataProviderDump
+restore(): void
+abstract update(): void
+setDataProvider(): void
+getSelection(): any
+setSelection(): void
+sortBy(): boolean
+abstract setHighlight(): boolean
+abstract getHighlight(): number
#listenersChanged(): void
#enableHighlightListening(): void
#addCopyListener(): void

RankingEvents

+static EVENT_WIDTH_CHANGED: "widthChanged"
+static EVENT_UPDATE_DATA: "updateData"
+static EVENT_RECREATE: "recreate"
+static EVENT_HIGHLIGHT_CHANGED: "highlightChanged"

+fire(): void
#createEventList(): string[]

EngineRanking

+static EVENT_WIDTH_CHANGED: "widthChanged"
+static EVENT_UPDATE_DATA: "updateData"
+static EVENT_RECREATE: "recreate"
+static EVENT_HIGHLIGHT_CHANGED: "highlightChanged"
-_context: ICellRenderContext<RenderColumn>
- loadingCanvas: any
- renderCtx: IRankingBodyContext
- data: (IGroupItem | Readonly<IOrderedGroup>)[]
- selection: SelectionManager
- highlight: number
-canvasPool: HTMLCanvasElement[]
-currentCanvasShift: number
-currentCanvasWidth: number
-events: RankingEvents
- roptions: Readonly<IEngineRankingOptions>
- delayedUpdate: (this: { type: string
}) => void
-delayedUpdateAll: () => void
-delayedUpdateColumnWidths: () => void
-columns: RenderColumn[]
-canvasMouseHandler: { timer: any
hoveredRows: any
cleanUp: () => void
enter: (evt: MouseEvent) => void
leave: (evt: HTMLElement | MouseEvent) => void
unhover: (row: HTMLElement) => void
}
- highlightHandler: { enabled: boolean
enter: (evt: MouseEvent) => void
leave: () => void
}
+ranking: Ranking
- ctx: IEngineRankingContext

+on(): this
#onVisibilityChanged(): void
+updateHeaders(): any
#createHeader(): any
- updateColumnSummaryFlag(): void
#updateHeader(): any
#createCell(): any
- createCellHandled(): HTMLElement
#updateCell(): any
-selectCanvas(): any
- rowFlags(): any
-visibleRenderedWidth(): number
- pushLazyRedraw(): void
- renderRow(): void
#updateCanvasCell(): void
- reindex(): void
+updateAll(): void
+updateBody(): void
+updateHeaderOf(): any
#createRow(): void
#updateRow(): void
- updateCanvasBody(): void
-toRowMeta(): string
- updateCanvasRule(): void
#updateShifts(): void
- recycleCanvas(): void
+enableHighlightListening(): void
- updateHoveredRow(): void
#forEachRow(): any
+updateSelection(): void
+updateColumnWidths(): void
- computeShiftedIndex(): { leftOffset: number
indexOffset: number
}
- updateColumn(): boolean
- updateCellImpl(): void
- initCellClasses(): void
+destroy(): void
+groupData(): (IGroupItem | Readonly<IOrderedGroup>)[]
+render(): any
+setHighlight(): boolean
+findNearest(): any
+scrollIntoView(): boolean
+getHighlight(): any
-createCol(): RenderColumn
-static isCanvasRenderedRow(): boolean
-static disableListener(): void

IEngineRankingContext

+createRenderer(): IRenderers

ITableSection

IRankingHeaderContextContainer

+idPrefix: string
+document: Document
+provider: IDataProvider
+dialogManager: DialogManager
+flags: ILineUpFlags
+caches: { toolbar: Map<string, IToolbarAction[]>
toolbarAddons: Map<string, IToolbarDialogAddon[]>
}

+asElement(): HTMLElement
+sanitize(): string
+resolveToolbarActions(): IToolbarAction[]
+resolveToolbarDialogAddons(): IToolbarDialogAddon[]
+getPossibleRenderer(): { item: IRenderInfo[]
group: IRenderInfo[]
summary: IRenderInfo[]
}
+summaryRenderer(): ISummaryRenderer

EngineRenderer

+static EVENT_HIGHLIGHT_CHANGED: "highlightChanged"
+static EVENT_DIALOG_OPENED: "dialogOpened"
+static EVENT_DIALOG_CLOSED: "dialogClosed"
#options: Readonly<ILineUpOptions>
+node: HTMLElement
- resizeObserver: ResizeObserver
-table: MultiTableRowRenderer
- rankings: EngineRanking[]
-slopeGraphs: SlopeGraph[]
+ctx: IRankingHeaderContextContainer & IRenderContext & IEngineRankingContext
- updateAbles: ((ctx: Readonly<IRankingHeaderContextContainer>) => void)[]
-zoomFactor: number
+idPrefix: any
-enabledHighlightListening: boolean
+selectionIndicator: SelectionIndicator
#data: ADataProvider

+zoomOut(): void
+zoomIn(): void
- updateZoomFactor(): void
+pushUpdateAble(): void
#createEventList(): string[]
+on(): this
+setDataProvider(): void
-takeDownProvider(): void
- initProvider(): void
- updateSelection(): void
- updateHist(): void
- addRanking(): void
- updateRotatedHeaderState(): void
- removeRanking(): void
+update(): void
- updateUpdateAbles(): void
- updateSlopeGraphs(): void
+setHighlight(): boolean
+setHighlightToNearest(): boolean
+getHighlight(): any
+enableHighlightListening(): void
+destroy(): void

IRankingBodyContext

+isGroup(): boolean
+getGroup(): Readonly<IOrderedGroup>
+getRow(): IGroupItem

LineUp

+static EVENT_SELECTION_CHANGED: "selectionChanged"
+static EVENT_DIALOG_OPENED: "dialogOpened"
+static EVENT_DIALOG_CLOSED: "dialogClosed"
+static EVENT_HIGHLIGHT_CHANGED: "highlightChanged"
- renderer: EngineRenderer
- panel: SidePanel
- options: ITaggleOptions

+destroy(): void
+update(): void
+setDataProvider(): void
+setHighlight(): boolean
+getHighlight(): any
#enableHighlightListening(): void

MultiLevelRenderColumn

-summaries: ISummaryRenderer[]

+createHeader(): HTMLElement | { item: HTMLElement
ready: IAbortAblePromise<void>
}
- matchChildren(): void
+updateHeader(): any
+hasSummaryLine(): boolean
+updateWidthRule(): any
- updateNested(): HTMLElement | { item: HTMLElement
ready: IAbortAblePromise<void>
}

RenderColumn

+renderers: IRenderers
+c: Column
+index: number
#ctx: Readonly<IRankingBodyContext>
#flags: ILineUpFlags

-singleRenderer(): HTMLElement
-groupRenderer(): HTMLElement
-summaryRenderer(): HTMLElement
+createHeader(): any
+hasSummaryLine(): boolean
+updateHeader(): any
+createCell(): any
+updateCell(): any
+renderCell(): any

IColumn

SelectionManager

+static EVENT_SELECT_RANGE: "selectRange"
-static MIN_DISTANCE: 10
- hr: HTMLHRElement
-selectionActivateFilter: string | boolean
- start: IPoint & IShift
- isFirefox: boolean
-ctx: { provider: IDataProvider
}
- body: HTMLElement

#createEventList(): string[]
+on(): this
-select(): void
-showHint(): void
+remove(): void
+add(): void
+selectRange(): void
+updateState(): void
+update(): void

ItemSlope

- left: number
- right: number
+dataIndices: number[]

+isSelected(): boolean
+update(): void

GroupSlope

- left: [number, number]
- right: [number, number]
+dataIndices: number[]

+isSelected(): boolean
+update(): void

SlopeGraph

+node: SVGSVGElement
- leftSlopes: ISlope[][]
- rightSlopes: ISlope[][]
- pool: SVGPathElement[]
- scrollListener: (act: { top: number
height: number
}) => void
+width: any
+height: 0
-current: { leftRanking: Ranking
left: (IGroupItem | Readonly<IOrderedGroup>)[]
leftContext: IExceptionContext
rightRanking: Ranking
right: (IGroupItem | Readonly<...>)[]
rightContext: IExceptionContext
}
-chosen: any
-chosenSelectionOnly: any
-_mode: EMode
+header: HTMLElement
+body: HTMLElement
+id: string
-ctx: IRankingHeaderContextContainer

+init(): void
- initHeader(): void
+hide(): void
+show(): void
+destroy(): void
+rebuild(): void
-computeSlopes(): void
- prepareRightSlopes(): any
- revalidate(): void
+highlight(): boolean
- onScrolledVertically(): void
- choose(): any
- chooseSelection(): any
- updatePath(): void
- render(): void
- addPath(): SVGPathElement
- matchLength(): SVGPathElement[]
+updateSelection(): void

ISlope

+dataIndices: number[]

+isSelected(): boolean
+update(): void

IBuilderAdapterColumnDescProps

+column: string
+asMap?: boolean
+asArray?: number | boolean | string[]
+custom?: { [key: string]: any
}

IBuilderAdapterCategoricalColumnDescProps

+asOrdinal?: boolean
+categories?: (string | Partial<ICategory>)[]
+missingCategory?: string | Partial<ICategory>
+asSet?: string | boolean

IBuilderAdapterDateColumnDescProps

+dateFormat?: string
+dateParse?: string

IBuilderAdapterHierarchyColumnDescProps

+hierarchy: IPartialCategoryNode
+hierarchySeparator?: string

IBuilderAdapterNumberColumnDescProps

+domain?: [number, number]
+range?: [number, number]
+mapping?: "linear" | "sqrt" | "pow1.1" | "pow2" | "pow3"
+scripted?: string
+sort?: EAdvancedSortMethod
+colorMapping?: string

IBuilderAdapterStringColumnDescProps

+editable?: boolean
+html?: boolean
+pattern?: string
+patternTemplates?: string[]

IBuilderAdapterBooleanColumnDescProps

+trueMarker?: string
+falseMarker?: string

IBuilderAdapterActionsColumnDescProps

+actions?: IAction[]
+groupActions?: IGroupAction[]

ActionsColumnBuilder

+action(): this
+actions(): this
+groupAction(): this
+groupActions(): this

ColumnBuilder<T>

#desc: T

+label(): this
+htmlLabel(): this
+hidden(): this
+summary(): this
+htmlSummary(): this
+description(): this
+frozen(): this
+renderer(): this
+custom(): this
+width(): this
+color(): this
+asArray(): this
+asMap(): this
+build(): T

BooleanColumnBuilder

+trueMarker(): this
+falseMarker(): this
+trueValues(): this

CategoricalColumnBuilder

+asOrdinal(): this
+categories(): this
+asSet(): this
- derive(): void
+build(): ICategoricalColumnDesc

DateColumnBuilder

+format(): this

HierarchyColumnBuilder

+hierarchy(): this
+build(): IHierarchyColumnDesc

NumberColumnBuilder

+mapping(): this
+color(): this
+colorMapping(): this
+numberFormat(): this
+scripted(): this
+asArray(): this
+asMap(): this
+asBoxPlot(): this
-derive(): [number, number]
+build(): INumberColumnDesc

StringColumnBuilder

+editable(): this
+alignment(): this
+html(): this
+pattern(): this

AddonDialog

- handlers: IToolbarDialogAddonHandler[]
-column: Column
- addons: IToolbarDialogAddon[]
-ctx: Readonly<IRankingHeaderContextContainer>
- onClick?: () => void

#build(): void
#submit(): boolean
#cancel(): void
#reset(): void

ADialog

- options: Readonly<IDialogOptions>
+node: HTMLFormElement
#floatingUiCleanup: () => void
#dialog: Readonly<IDialogContext>

#abstract build(): boolean | void
#showLivePreviews(): boolean
#enableLivePreviews(): void
+equals(): boolean
#appendDialogButtons(): void
+open(): void
#triggerSubmit(): boolean
#find(): T
#findInput(): HTMLInputElement
#forEach(): T[]
#abstract reset(): void
#abstract submit(): boolean
#abstract cancel(): void
+cleanUp(): void
#destroy(): void

APopup

#submit(): boolean
#reset(): void
#cancel(): void

CategoricalColorMappingDialog

- before: ICategoricalColorMappingFunction
-column: ICategoricalColumn | CategoricalMapColumn | CategoricalsColumn

#build(): void
+reset(): void
+submit(): boolean
+cancel(): void

CategoricalFilterDialog

- before: ICategoricalFilter
-column: BooleanColumn | CategoricalColumn | CategoricalsColumn | SetColumn
-ctx: Readonly<IRankingHeaderContextContainer>

#build(): void
- updateStats(): void
- updateFilter(): void
#reset(): void
#cancel(): void
#submit(): boolean
+cleanUp(): void

CategoricalMappingFilterDialog

- before: ICategoricalFilter
-column: OrdinalColumn

#build(): void
- updateFilter(): void
#cancel(): void
#reset(): void
#submit(): boolean

ChangeRendererDialog

- before: { renderer: string
group: string
summary: string
}
-column: Column
-ctx: Readonly<IRankingHeaderContextContainer>

#build(): void
#cancel(): void
#reset(): void
#submit(): boolean

ChooseRankingDialog

- items: HTMLElement[]

#build(): void

ColorMappingDialog

- before: IColorMappingFunction
- id: string
-column: IMapAbleColumn
-ctx: Readonly<IRankingHeaderContextContainer>

- createTemplate(): string
- applyColor(): void
#build(): void
- render(): void
#reset(): void
#submit(): boolean
#cancel(): void
- updateGradients(): void

CompositeChildrenDialog

- id: string
-column: CompositeColumn
-ctx: Readonly<IRankingHeaderContextContainer>

+cleanUp(): void
#build(): void

CutOffHierarchyDialog

- innerNodes: ICategoryInternalNode[]
- innerNodePaths: string[]
- before: ICutOffNode
-column: HierarchyColumn
-ctx: Readonly<IRankingHeaderContextContainer>

#build(): void
#reset(): void
#cancel(): void
#submit(): boolean

DateFilterDialog

- before: INumberFilter
- handler: { reset: () => void
submit: () => void
cleanUp: () => void
}
-column: DateColumn
-ctx: Readonly<IRankingHeaderContextContainer>

#build(): void
+cleanUp(): void
#reset(): void
#submit(): boolean
#cancel(): void

DialogManager

+static EVENT_DIALOG_OPENED: "dialogOpened"
+static EVENT_DIALOG_CLOSED: "dialogClosed"
-escKeyListener: (evt: KeyboardEvent) => void
- openDialogs: ADialog[]
+node: HTMLElement
+livePreviews: Partial<ILivePreviewOptions>
+onDialogBackgroundClick: "cancel" | "confirm"

#createEventList(): string[]
+on(): this
+setHighlight(): void
+setHighlightColumn(): void
+clearHighlight(): void
- removeLast(): void
+removeAll(): void
+triggerDialogClosed(): void
+remove(): boolean
+removeAboveLevel(): void
+removeLike(): boolean
-setUp(): void
-takeDown(): void
+push(): void

EditPatternDialog

- before: string
-column: LinkColumn | LinkMapColumn | LinksColumn
-ctx: Readonly<IRankingHeaderContextContainer>

#build(): void
#cancel(): void
#reset(): void
#submit(): boolean

GroupDialog

- handlers: IToolbarDialogAddonHandler[]
-column: Column
-ctx: Readonly<IRankingHeaderContextContainer>

#build(): void
#submit(): boolean
#cancel(): void
#reset(): void

InputDateDialog

- ioptions: Readonly<IInputDateOptions>
-callback: (value: Date) => void

#build(): void
+submit(): boolean

InputNumberDialog

- ioptions: Readonly<IInputNumberOptions>
-callback: (value: number) => void

#build(): void
+submit(): boolean

MappingDialog

-scale: IMappingFunction
- mappingLines: MappingLine[]
- rawDomain: [number, number]
-data: Promise<ISequence<number>>
- idPrefix: string
- before: IMappingFunction
- mappingAdapter: IMappingAdapter
-column: IMapAbleColumn

+build(): void
- updateDomainWarning(): void
-createMappings(): void
- update(): void
- updateLines(): void
#reset(): void
-copyMapping(): void
- normalizeRaw(): number
- unnormalizeRaw(): number
-computeScale(): ScaleMappingFunction | ScriptMappingFunction
#submit(): boolean
#cancel(): void

MappingLineDialog

- before: { domain: number
range: number
}
- line: { destroy(): void
domain: number
range: number
frozen: boolean
update(domain: number, range: number, trigger: boolean): void
}
-adapter: IMappingAdapter

+build(): void
+cleanUp(): void
#cancel(): void
#reset(): void
#submit(): boolean

MoreColumnOptionsDialog

-column: Column
- mode: "header" | "sidePanel"
-ctx: Readonly<IRankingHeaderContextContainer>

#build(): void

MoreRankingOptionsDialog

- ranking: Ranking
-ctx: Readonly<IRankingHeaderContextContainer>

- addIcon(): void
#build(): void

NumberFilterDialog

- before: INumberFilter
- handler: { reset: () => void
submit: () => void
cleanUp: () => void
}
-column: IMapAbleColumn
-ctx: Readonly<IRankingHeaderContextContainer>

+build(): void
+cleanUp(): void
#reset(): void
#submit(): boolean
#cancel(): void

ReduceDialog

- handler: IToolbarDialogAddonHandler
-column: ReduceColumn

#build(): void
#submit(): boolean
#reset(): void
#cancel(): void

RenameDialog

- before: IColumnMetaData
-column: Column

#build(): void
#reset(): void
#submit(): boolean
#cancel(): void

RenameRankingDialog

- before: string
- ranking: Ranking

#build(): void
#reset(): void
#cancel(): void
#submit(): boolean

ScriptEditDialog

- before: string
-column: ScriptColumn

#build(): void
#cancel(): void
#reset(): void
#submit(): boolean

SearchDialog

- current: { filterType: EStringFilterType
search: string | RegExp
indices: number[]
index: number
}
- column: Column
- provider: IDataProvider

#build(): void
-jumpToPreviousResult(): void
-jumpToNextResult(): void
- searchAndJump(): boolean
#submit(): boolean
#reset(): void
#cancel(): void

SelectionFilterDialog

- before: number[]
-column: SelectionColumn
-ctx: Readonly<IRankingHeaderContextContainer>

- updateFilter(): void
#reset(): void
#cancel(): void
#submit(): boolean
-guessRemoveFilter(): string
#build(): void

ShowTopNDialog

- before: number
- provider: IDataProvider

#build(): void
#cancel(): void
#submit(): boolean
#reset(): void

SortDialog

- handlers: IToolbarDialogAddonHandler[]
-column: Column
-groupSortBy: boolean
-ctx: Readonly<IRankingHeaderContextContainer>

#build(): void
#submit(): boolean
#cancel(): void
#reset(): void

StringFilterDialog

- before: IStringFilter
-column: StringColumn
-ctx: Readonly<IRankingHeaderContextContainer>

- updateFilter(): void
#reset(): void
#cancel(): void
#submit(): boolean
#build(): void

WeightsEditDialog

-weights: number[]
-column: StackColumn

#cancel(): void
#reset(): void
#build(): void
- updateBar(): void
- distributeWeights(): void
#appendDialogButtons(): void
+submit(): boolean

SearchBox<T>

+static EVENT_SELECT: "select"
-static SEARCH_ITEM_SELECTOR: string
- options: Readonly<ISearchBoxOptions<T>>
+node: HTMLElement
- search: HTMLInputElement
- body: HTMLElement
- itemTemplate: HTMLElement
-groupTemplate: HTMLElement
-values: (T | IGroupSearchItem<T>)[]

- buildDialog(): void
- handleKey(): void
-select(): void
+focus(): void
- highlightNext(): void
- highlightPrevious(): void
- blur(): void
-filter(): void
-filterResults(): any
#createEventList(): string[]
+on(): this

Taggle

+static EVENT_SELECTION_CHANGED: "selectionChanged"
+static EVENT_DIALOG_OPENED: "dialogOpened"
+static EVENT_DIALOG_CLOSED: "dialogClosed"
+static EVENT_HIGHLIGHT_CHANGED: "highlightChanged"
- spaceFilling: HTMLElement
- renderer: TaggleRenderer
- panel: SidePanel
- spaceFillingRule: IRule
- options: ITaggleOptions

+isOverviewMode(): boolean
+setOverviewMode(): void
- updateLodRules(): void
- setViolation(): void
+destroy(): void
+update(): void
+setHighlight(): boolean
+getHighlight(): any
#enableHighlightListening(): void
+setDataProvider(): void

TaggleRenderer

+static EVENT_HIGHLIGHT_CHANGED: "highlightChanged"
+static EVENT_DIALOG_OPENED: "dialogOpened"
+static EVENT_DIALOG_CLOSED: "dialogClosed"
- isDynamicLeafHeight: boolean
- rule: IRule
- levelOfDetail: (rowIndex: number) => "high" | "low"
- renderer: EngineRenderer
- options: Readonly<ITaggleRendererOptions>
+data: ADataProvider

+pushUpdateAble(): void
-dynamicHeight(): { defaultHeight: number
height: (item: IGroupItem | Readonly<IOrderedGroup>) => number
padding: number
}
#createEventList(): string[]
+on(): this
+zoomOut(): void
+zoomIn(): void
+switchRule(): void
+getRule(): IRule
+destroy(): void
+update(): void
+setDataProvider(): void
+setHighlight(): boolean
+getHighlight(): any
+enableHighlightListening(): void